

Yazılım Kalite Güvencesi

Dönem Projesi Aşama 1: Geliştirme Raporu

Task Manager API

Furkan Öztürk
Kocaeli Üniversitesi
Yazılım Mühendisliği
230229083

Taha Yasin Çiçek
Kocaeli Üniversitesi
Yazılım Mühendisliği
230229088

Betül Fikriye Turan
Kocaeli Üniversitesi
Yazılım Mühendisliği
220229073

Elmas Güneş
Kocaeli Üniversitesi
Yazılım Mühendisliği
220229054

Özet—Bu rapor, Yazılım Kalite Güvencesi dersi kapsamında geliştirilen Task Manager REST API uygulamasının Aşama 1 geliştirme sürecini belgelemektedir. Uygulama; TypeScript, Node.js ve Express framework kullanılarak inşa edilmiş olup kullanıcı kimlik doğrulama, rol tabanlı yetkilendirme, görev yönetimi CRUD işlemleri, proje ve etiket yönetimi gibi özellikler sunmaktadır. Test altyapısı Vitest ve Supertest üzerine kurulmuş, toplam 98 test (31 smoke + 67 unit) başarıyla geçmektedir.

Index Terms—REST API, TypeScript, Express, SQLite, Yazılım Kalite Güvencesi, Birim Testi, Duman Testi

I. GİRİŞ

Bu projede, yazılım kalite güvencesi prensipleri doğrultusunda çalışan bir web uygulaması geliştirilmesi ve gereksinimlerin testlerle kanıtlanması hedeflenmiştir. Seçilen uygulama, kullanıcıların görev oluşturmaya, güncellemesine, listelemesine ve silmesine olanak tanıyan bir görev yönetimi (Task Manager) REST API'dir. Kimlik doğrulama ve yetkilendirme mekanizmaları da dahil olmak üzere tam işlevsel bir backend servisi sunulmaktadır. Ek olarak, bonus kapsamında React + Vite ile geliştirilmiş bir kullanıcı arayüzü de projeye dahil edilmiştir.

II. MİMARİ ÖZET

Uygulama, katmanlı mimari (layered architecture) prensibine göre tasarlanmıştır. Bu bölümde her katmanın sorumluluğu ve teknik detayları açıklanmaktadır.

A. Sunum Katmanı

Sunum katmanında Express Router'ları ile tanımlanmış HTTP endpoint'leri bulunmaktadır. Her kaynak türü (auth, tasks, projects, labels, comments, admin, reports) ayrı bir route dosyasında modüler biçimde organize edilmiştir.

B. İş Mantığı Katmanı

İş mantığı, `services/` dizini altında ayrı servis modüllerinde konumlandırılmıştır. Bu yaklaşım, kodun test edilebilirliğini ve bakımını önemli ölçüde kolaylaştırmaktadır. Servis fonksiyonları doğrudan veritabanı nesnesi olarak route handler'lardan bağımsız çalışabilmektedir.

C. Veri Katmanı

Veritabanı katmanında `better-sqlite3` kütüphanesi kullanılarak senkron SQLite operasyonları gerçekleştirilmektedir. `db.ts` modülü uygulama başladığında tabloları otomatik olarak oluşturmaktadır. WAL (Write-Ahead Logging) modu etkinleştirilerek eş zamanlı okuma performansı optimize edilmiş, yabancı anahtar kısıtları da açıkça aktif hale getirilmiştir.

D. Veritabanı Tercih

Veritabanı olarak SQLite (`better-sqlite3`) seçilmiştir. Bu tercih, uygulamanın bağımsız çalışabilmesini (ek veritabanı sunucusu gerektirmemesini) sağlamakta ve özellikle geliştirme ortamında kurulum karmaşıklığını ortadan kaldırmaktadır. `better-sqlite3` kütüphanesi senkron API sunduğu için kod okunabilirliği artmakta ve `async/await` karmaşası yaşanmamaktadır.

E. Kimlik Doğrulama ve Güvenlik

Kimlik doğrulama, `express-session` middleware'i aracılığıyla sunucu tarafı oturum yönetimiyle sağlanmaktadır. JWT yerine sunucu tarafı oturum tercih edilmesinin temel sebebi güvenlik odaklıdır: oturum verileri sunucuda tutulduğundan, bir oturumu geçersiz kılmak anlık olarak mümkündür. JWT tabanlı yaklaşımda ise token süresi dolana kadar geçersiz kılma mümkün değildir.

Oturum kimliği `httpOnly` özelliğine sahip bir çerezde saklanarak JavaScript tarafından erişilmesi engellenmekte, böylece XSS saldırılarına karşı ek koruma sağlanmaktadır. Kullanıcı şifreleri veritabanına kaydedilmeden önce `bcrypt` ile hash'lenerek güvenlik güçlendirilmiştir.

III. KULLANILAN TEKNOLOJİLER

Tablo I'de projede kullanılan teknolojiler ve gerekçeleri özetlenmektedir.

IV. KURULUM VE ÇALIŞTIRMA

A. Gereksinimler

- Node.js $\geq 18.x$
- npm $\geq 9.x$

Tablo I
TEKNOLOJILER

Katman	Teknoloji
Dil	TypeScript 5.3
Runtime	Node.js (LTS)
Backend (API)	Express 4.18
Auth	express-session + httpOnly cookie
Veritabanı	SQLite (better-sqlite3)
Şifre Hash	bcrypt
Test Framework	Vitest 1.2 + Supertest 6.3
UI (Bonus)	React 18 + Vite 5 + TailwindCSS

B. Adımlar

```
# Bağımlilikler yükle
npm install

# Veritabanını başlangıç verileriyle doldur
npm run seed

# Geliştirme modunda başlat
npm run dev

# Üretim için derle ve çalıştır
npm run build
npm start
```

C. Varsayılan Kullanıcılar (Seed)

npm run seed komutu çalıştırıldıktan sonra Tablo II'deki kullanıcılar ve 3 örnek görev oluşturulur.

Tablo II
VARSAYILAN KULLANICILAR

Kullanıcı Adı	Şifre	Rol
admin	Admin123!	admin
user1	User123!	user

V. API ENDPOINT DOKÜMANTASYONU

A. Kimlik Doğrulama

Tablo III
AUTH ENDPOINT'LERİ

Metod	Endpoint	Açıklama	Yetki
POST	/api/auth/register	Yeni kullanıcı kaydı	Herkese açık
POST	/api/auth/login	Giriş yap	Herkese açık
POST	/api/auth/logout	Oturumu sonlandır	Giriş yapmış
GET	/api/auth/me	Oturum bilgisi	Giriş yapmış

1) Kayıt İsteği

```
POST /api/auth/register
{
  "username": "kullanici",
  "email": "kullanici@ornek.com",
  "password": "Sifre123!"
}
```

2) Giriş İsteği

```
POST /api/auth/login
{
  "username": "kullanici",
  "password": "Sifre123!"
}
```

B. Görevler (Tasks)

Tüm görev endpoint'leri kimlik doğrulaması gerektirir.

Tablo IV
TASK ENDPOINT'LERİ

Metod	Endpoint	Açıklama
GET	/api/tasks	Tüm görevleri listele
POST	/api/tasks	Yeni görev oluştur
GET	/api/tasks/:id	Tek görev getir
PATCH	/api/tasks/:id	Görevi güncelle
DELETE	/api/tasks/:id	Görevi sil

1) Görev Oluşturma

```
POST /api/tasks
{
  "title": "Görev başlığı",
  "description": "İsteğe bağlı açıklama",
  "status": "todo"
}
```

Geçerli status değerleri: todo, in-progress, done. Geçerli priority değerleri: low, medium, high, urgent.

C. Admin Endpoint'leri

Yalnızca admin rolündeki kullanıcılar erişebilir.

Tablo V
ADMIN ENDPOINT'LERİ

Metod	Endpoint	Açıklama
GET	/api/admin/users	Tüm kullanıcıları listele
GET	/api/admin/stats	İstatistikler

D. Hata Yanıtları ve HTTP Durum Kodları

API, doğrulama hataları için alan bazlı hata mesajları döndürmektedir:

```
{
  "errors": [
    { "field": "title", "message": "Title is required" }
  ]
}
```

Tablo VI'de kullanılan HTTP durum kodları listelenmiştir.

VI. TESTLER

Bu projede kalite güvencesi kapsamında iki temel test seviyesi uygulanmıştır: Duman Testleri (Smoke Tests) ve Birim Testleri (Unit Tests).

Tablo VI
HTTP DURUM KODLARI

Kod	Açıklama
200	Başarılı
201	Kaynak oluşturuldu
204	Başarılı, içerik yok (silme)
400	Geçersiz istek / doğrulama hatası
401	Kimlik doğrulaması gerekli
403	Yetkisiz erişim
404	Kaynak bulunamadı
409	Çakışma (kullanıcı adı/e-posta mevcut)
500	Sunucu hatası

A. Test Komutları

```
# Tüm testleri çalıştır
npm test

# Yalnızca duman testlerini çalıştır
npm run test:smoke

# Yalnızca birim testlerini çalıştır
npm run test:unit
```

B. Duman Testleri (Smoke Tests)

Duman testleri, uygulamanın en kritik işlevlerinin uçtan uca doğru çalışıp çalışmadığını doğrular. tests/smoke/api.smoke.test.ts içinde yer alan 31 senaryo aşağıdaki başlıkları kapsamaktadır:

- 1) **Kimlik Doğrulama:** Kayıt olma (201), giriş yapma (200) ve oturum çerezinin doğrulanması.
- 2) **Yetkilendirme:** Oturum açmadan korumalı endpoint'lere erişimin engellenmesi (401) ve admin-only endpoint testleri (user → 403, admin → 200).
- 3) **Ana Varlık (Task) CRUD:** Görev oluşturma (201), listeleme (200), detay görüntüleme (200), güncelleme (200) ve silme (204, ardından 404).
- 4) **Doğrulama (Validation):** Geçersiz verilerle yapılan isteklerin 400 hatası ile reddedilmesi.
- 5) **Ek Özellikler:** Yorum ekleme/silme, etiket yönetimi, alt görev (subtask) oluşturma ve proje bazlı yönetim.
- 6) **Raporlama:** İstatistik ve özet raporlarına yetkili/yetkisiz erişim.

Tablo VII'de zorunlu 10 smoke test senaryosunun karşılanma durumu gösterilmiştir.

C. Birim Testleri (Unit Tests)

Birim testleri, servis katmanındaki iş kurallarını ve veri doğrulama mantığını in-memory veritabanı ile izole bir şekilde test etmektedir. Toplam 67 birim testi aşağıdaki başlıkları içermektedir:

- **Validation / İş Kuralları:** taskService, authService, labelService, commentService, projectService içindeki girdi doğrulama fonksiyonları (boş değer, uzunluk, format kontrolü).

Tablo VII
ZORUNLU SMOKE TEST SENARYOLARI

#	Senaryo	Durum
1	Register → 201	Geçti
2	Login → 200 (cookie doğrulandı)	Geçti
3	Auth gerekli endpoint → 401	Geçti
4	Create (POST) → 201	Geçti
5	Read-list (GET) → 200	Geçti
6	Read-single (GET/:id) → 200	Geçti
7	Update (PATCH) → 200	Geçti
8	Delete → 204, ardından 404	Geçti
9	Validation → 400	Geçti
10	Admin-only: user→403, admin→200	Geçti

- **Yetkilendirme Kararı:** authService içindeki rol bazlı (admin/user) erişim izinleri (isAdmin, canAccess).
- **Hata Durumları:** Kaynak bulunamadı (404), yetkisiz erişim (403) ve çakışma durumlarının servis seviyesinde yönetimi.
- **Servis Mantığı:** Ana varlık ve yardımcı varlıkların (etiket, yorum, proje) veritabanı ile olan CRUD etkileşimleri ve karmaşık istatistik hesaplama algoritmaları (reportService).

Tablo VIII'de birim testlerinin dosya bazlı dağılımı gösterilmiştir.

Tablo VIII
BİRİM TEST DAĞILIMI

Test Dosyası	Test Sayısı
taskService.test.ts	24
authService.test.ts	10
labelService.test.ts	9
commentService.test.ts	8
reportService.test.ts	8
projectService.test.ts	8
Toplam	67

D. Test Logları

Aşağıda npm test komutunun çıktısı yer almaktadır:

```
Test Files 7 passed (7)
Tests      98 passed (98)
Start at   02:20:15
Duration   1.53s (transform 213ms, setup 0ms,
collect 651ms, tests 510ms,
environment 0ms, prepare 149ms)
```

VII. OPSİYONEL UI (BONUS)

React 18 + Vite 5 + TailwindCSS ile geliştirilmiş bir kullanıcı arayüzü client/ dizininde bulunmaktadır.

A. UI Kurulumu

```
# Client bağımliliklerini yükle (proje kokunden)
npm run client:install

# Client'i geliştirme modunda baslat
npm run client:dev
```

Tarayıcıda `http://localhost:5173` adresinden erişilebilir. Backend (port 3000) çalışıyor olmalıdır.

B. UI Özellikleri

- Giriş ve kayıt sayfaları
- Dashboard: genel istatistikler, yaklaşan görevler, öncelik dağılımı
- Görev yönetimi: Kanban board, oluşturma, düzenleme, silme, etiketleme
- Proje yönetimi: oluşturma, üye ekleme/çıkarma
- Raporlama: takım özeti, kullanıcı bazlı istatistikler, proje ilerleme durumu
- Admin paneli: kullanıcı listesi, sistem istatistikleri (yalnızca admin rolü)
- Karanlık/aydınlık tema desteği
- Responsive tasarım (mobil uyumlu)

VIII. SONUÇ

Bu projede, yazılım kalite güvencesi prensipleri doğrultusunda katmanlı mimariye sahip bir Task Manager REST API geliştirilmiştir. Uygulama; kimlik doğrulama, rol tabanlı yetkilendirme, CRUD operasyonları ve sunucu tarafı doğrulama gibi tüm zorunlu gereksinimleri karşılamaktadır. İki katmanlı test stratejisi (31 smoke + 67 unit = 98 test) hem bireysel bileşenlerin hem de sistemin bütününe doğru çalıştığını garanti etmektedir. Bonus olarak sunulan React arayüzü, API'nin tüm özelliklerini kullanıcı dostu bir şekilde erişilebilir kılmaktadır.